**Innovative Evaluation Practices for BCSE308P**
**Computer Networks Lab**

**Fall Semester 2026–27**

# AI ASSISTED SELF HEALING SOFTWARE DEFINED NETWORK FOR PATH DEGRADATION DETECTION AND RECOVERY


| Registration number | Student name |
|---|---|
| 24BCE2052 | VAIBHAV BAKSHI |


**B.Tech.**
**in**
**Computer Science and Engineering**

**School of Computer Science and Engineering**




October 2026

## ABSTRACT

This project demonstrates automatic traffic rerouting in a small software-defined network. Mininet creates the network, Open vSwitch forwards packets, and a Python control loop measures the primary and backup routes. A local browser dashboard shows the selected path, live measurements, recorded playback, and saved evidence.

This report uses eight current demonstrations recorded on 27 October 2026. Six scenarios use the default random forest with a measured-loss guard. Two additional packet-loss runs compare the forest alone and fixed rules. All runs use six switches, 4 Mbps application traffic, 10 Mbps links, and 36 seconds of traffic. Each condition ran once.

The guarded controller verified recovery in the link-failure, packet-loss, congestion, and delay runs. Normal operation needed no reroute. With both paths disconnected, no healthy alternative existed. In the packet-loss comparison, guarded RF and fixed rules restored communication, while RF alone made no route change.

The results explain the current implementation rather than reproduce the earlier research campaigns. Tables distinguish data received, application packet loss, route-change time, and recovery confirmation. Path probes remain separate from receiver-side UDP statistics. These single-run observations show how this testbed behaved and do not establish accuracy or consistent timing across other networks.

# CERTIFICATE

This is to certify that the project report titled “AI Assisted Self Healing Software Defined Network for Path Degradation Detection and Recovery” is a bonafide record of the work done by the student named below in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P Computer Networks Lab during the Fall Semester 2026–2027.

| **Name and registration number** | **Signature** |
| --- | --- |
| VAIBHAV BAKSHI 24BCE2052 | |

# TABLE OF CONTENTS


| Contents | Page |
|---|---|
| ABSTRACT | 2 |
| CERTIFICATE | 3 |
| 1 INTRODUCTION | 5 |
| 2 LITERATURE SURVEY | 6 |
| 3 NETWORK AND RECOVERY DESIGN | 7 |
| 4 IMPLEMENTATION AND MEASUREMENTS | 8 |
| 5 EXPERIMENT CONDITIONS | 9 |
| 6 CURRENT SCENARIO RESULTS | 10 |
| 7 RECOVERY TIMES EXPLAINED | 11 |
| 8 CURRENT METHOD COMPARISON | 12 |
| 9 READING THE DASHBOARD GRAPHS | 13 |
| 10 MODEL RELIABILITY | 14 |
| 11 CONCLUSION AND NEXT TESTS | 15 |
| 12 REFERENCES | 16 |
| APPENDIX A DEMONSTRATION GUIDE | 17 |
| APPENDIX B CURRENT RUN EVIDENCE | 18 |

# 1 INTRODUCTION

## 1.1 The problem

A sender can lose communication when its current network path fails, even when a second route still works. This project measures both routes and changes forwarding rules when the selected path becomes unhealthy and the alternative is usable. Healing means moving traffic around the problem. The controller does not physically repair the failed link.

## 1.2 What the demonstration answers

The current tests ask whether traffic can continue on the backup, how long the change and recovery checks take, and how much application data reaches the receiver. A separate packet-loss comparison asks what changes when the same forest runs with or without its loss safeguard, and how fixed rules behave under the same configuration.

## 1.3 Scope of this report

The main results are the eight demonstrations recorded on 27 October 2026. Each condition has one run. The older campaign means, repetition counts, and earlier-model accuracy are not used as current results. Previous report versions retain that historical material. This report uses the bundled detector, current runner, and saved evidence from the present dashboard.

## 1.4 Networking concepts demonstrated

| Concept | Where it appears |
|---|---|
| Forwarding | OpenFlow rules select the application branch. |
| Monitoring | Separate probes measure both routes and the application destination. |
| Fault tolerance | The backup carries traffic after a successful route change. |
| Performance | RTT, packet loss, link use, and receiver throughput describe different aspects of service. |

# 2 LITERATURE SURVEY

## 2.1 Technical background

Mininet provides an emulated network that runs real networking software on one host [1]. SDN separates forwarding from the control logic that changes network behavior [2]. Self-healing mechanisms and learned recovery decisions are established research subjects [3], [4]. The present project uses these ideas in a small observable laboratory.

## 2.2 Relationship to the current implementation

Open vSwitch forwards packets using OpenFlow 1.3 rules. The Python control loop measures conditions and updates those rules. The implementation does not require a separate Ryu or ONOS controller. Mininet and Open vSwitch documentation describe the underlying tools [5], [6].

## 2.3 Contribution and limits

The contribution is an implemented demonstration linking measurements, fault injection, route changes, and post-change verification. Independent probes keep the unused path observable. The website makes these steps visible and exposes the raw evidence. The loss guard is an explicit rule around a fitted model, rather than a new learning algorithm.

The current eight runs support a functional explanation of this implementation. They do not establish research novelty, statistical superiority over other controllers, or reliability on arbitrary networks.

# 3 NETWORK AND RECOVERY DESIGN

The default six-switch topology provides an upper primary branch and a lower backup branch. Both connect the same sender h1 and receiver h2. Background traffic hosts participate in congestion tests but are omitted from the diagram.

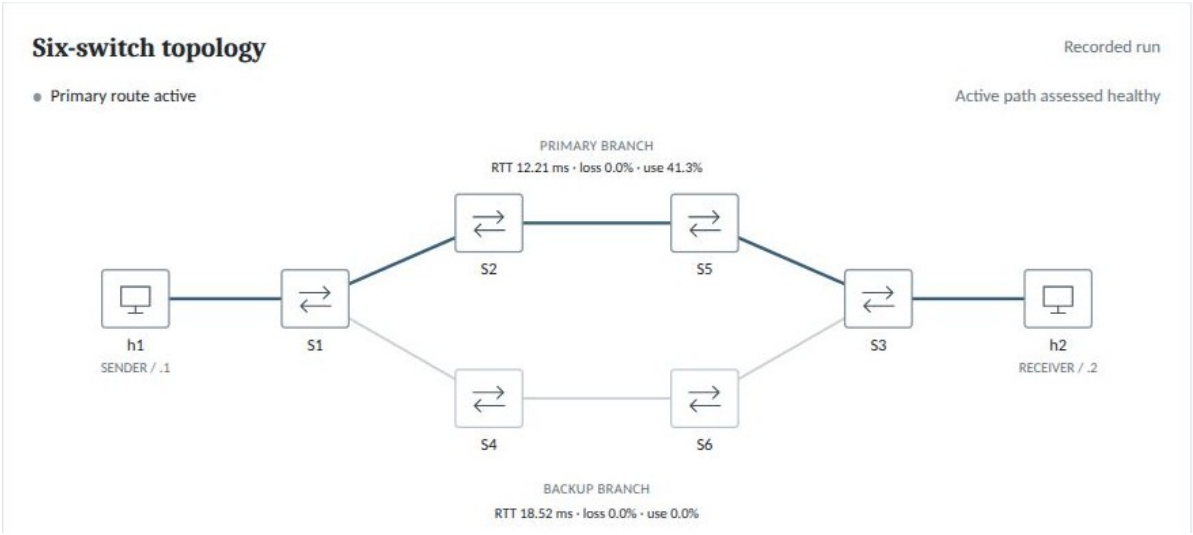



Figure 1 Current six-switch network during normal operation

## 3.1 The two application routes

The primary route uses h1, S1, S2, S5, S3, and h2. The backup route uses h1, S1, S4, S6, S3, and h2. Each inter-switch link has a configured 10 Mbps capacity in these saved runs. New experiments expose link capacity in the dashboard and through --capacity (greater than 0 and at most 100 Mbps; default 10). The project also supports a four-switch layout, but the results in this report all use six switches.

## 3.2 The recovery sequence

The controller measures both branches, checks whether the selected path is unhealthy, and checks whether the other path is usable. It then installs and verifies the application forwarding rules before committing the route change. Later application-destination probes determine whether recovery can be confirmed while the fault persists.

Soft degradation normally needs two consecutive unhealthy readings and observes a five-second hold-down between changes. Hard link failures bypass those waits. A known-down backup is never selected. These recorded runs kept traffic on backup after primary recovery. Current software also supports stable automatic return.

# 4 IMPLEMENTATION AND MEASUREMENTS

## 4.1 Software components

| Component | Role |
|---|---|
| Mininet | Creates hosts, virtual links, and switches. |
| Open vSwitch | Forwards real packets using OpenFlow rules. |
| Python runner | Starts traffic, injects faults, measures paths, and changes routes. |
| RF and loss guard | Judge path health from observations and the explicit loss rule. |
| Local dashboard | Shows settings, measurements, playback, comparison, and evidence files. |

## 4.2 Three probe streams

One set of pings stays on the primary branch, a second stays on the backup, and a third follows the current application route. This explains why the backup has an RTT even before it carries application traffic. Application probe RTT is a ping measurement to the application destination, rather than the timing of individual UDP packets.

## 4.3 Traffic and link utilization

The 4 Mbps setting is the offered UDP payload rate. Branch traffic is estimated from transmit-byte counter changes on the final inter-switch link of each branch, using the busier direction. Utilization divides that rate by the configured capacity. A 4.11 Mbps reading gives 41.1% of a 10 Mbps link. Headers, probes, and interval estimation can explain differences from the offered payload rate.

The receiver throughput and application loss are whole-run UDP statistics. The runner obtains these after traffic ends. They must not be confused with the latest path-probe window, estimated branch traffic, or per-switch latency, which the interface does not measure.

# 5 EXPERIMENT CONDITIONS

Eight distinct current runs form the evidence set. Six runs exercise the default guarded detector, one for each scenario. Two additional loss runs test RF only and fixed rules. The guarded loss run is shared by the scenario table and the method comparison; it is counted once.

| Setting | Current value |
|---|---|
| Topology | Six-switch extended diamond |
| Offered application traffic | 4 Mbps UDP |
| Configured branch link capacity | 10 Mbps |
| Traffic duration | 36 seconds, plus setup and cleanup |
| Requested fault timing | Around second 10 to second 26 |
| Default detector | Random Forest with the 15% measured-loss guard |
| Repetitions | One run per condition |

## 5.1 Conditions injected by the runner

Link failure disconnects one primary link. Packet loss configures 20% loss in each direction of one primary link. Increased delay configures 60 ms in each direction of that link. Congestion offers 12 Mbps of extra background traffic to a 10 Mbps branch. Both-down disconnects both branches. Normal operation injects no fault. For new CLI and dashboard runs, congestion background traffic is 1.2 times the selected capacity, retaining 12 Mbps at the default 10 Mbps.

## 5.2 Actual events and rounded results

Graphs shade the actual fault window recorded in events.jsonl. Setup and probe waits can move events away from the requested schedule. For example, the link-failure run cleared its fault at about 28 seconds. The values in the main tables are rounded to two decimals. Full precision remains in the run files listed in Appendix B.

## 5.3 What the results can establish

These are measured demonstrations rather than averages or repeated-run estimates. A successful execution means the runner completed and saved evidence. Successful recovery additionally requires the defined application-probe check. A missed recovery is reported as not observed, never as zero seconds.

# 6 CURRENT SCENARIO RESULTS

Read the last column first. It states whether the default guarded controller restored service in that run. Data received and packets lost describe the whole 36-second application session, including the periods before and after the fault.

| Scenario | Data received | Packets lost | What happened |
|---|---|---|---|
| Normal | 4.00 Mbps | 0.00% | No fault to recover from |
| Link failure | 3.76 Mbps | 5.86% | Restored in 8.02 s |
| Packet loss | 3.97 Mbps | 0.64% | Restored in 3.30 s |
| Congestion | 3.89 Mbps | 2.81% | Restored in 4.98 s |
| Increased delay | 4.00 Mbps | 0.01% | Restored in 3.33 s |
| Both paths down | 2.00 Mbps | 50.02% | No healthy alternative |

### 6.1 Normal and recoverable cases

Normal operation delivered about the requested 4 Mbps with no observed UDP loss and no route change. Link failure, packet loss, congestion, and increased delay each switched to the backup and verified recovery while their fault was active. The link-failure run had the largest whole-run packet loss among those recovered cases.

### 6.2 Both paths down

The both-down run delivered about 2 Mbps averaged over the session and lost about half the UDP packets. This does not mean the broken network carried 2 Mbps throughout the fault. The average includes healthy periods before and after the disconnection. No healthy branch existed during the fault, so rerouting could not restore service then.

### 6.3 Meaning of the delay result

The delay row describes guarded RF, not RF alone. Recovery can reflect the loss safeguard or the learned response to the measured combination of conditions. This one row does not establish that the forest reliably detects every delay-only fault.

# 7 RECOVERY TIMES EXPLAINED

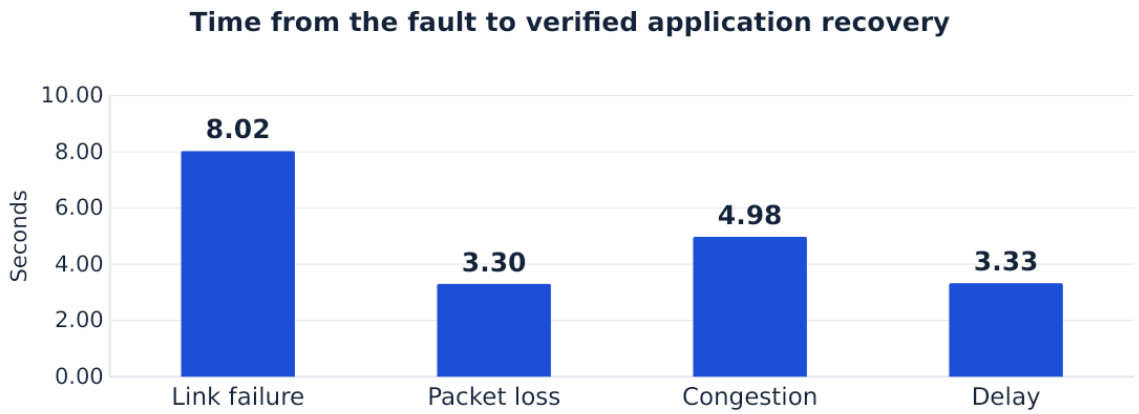



Figure 2 Current recovery confirmation times for four guarded demonstrations

A shorter bar means the runner confirmed recovery sooner in that particular run. Normal operation is omitted because no recovery was needed. Both-down is omitted because no reroute recovery was observed. Neither missing result is a zero-second recovery.

### 7.1 A route change is an action

In the link-failure run, the fault began at 10.01 seconds and the route changed at 12.11 seconds. The controller therefore took about 2.10 seconds from fault onset to the recorded forwarding change. Installing the rules itself took about 0.013 seconds; observation and decision timing make the total longer.

### 7.2 Verified recovery is a later check

The runner confirmed recovery 8.02 seconds after fault onset. It required two consecutive post-change application probe windows with zero observed loss and RTT below 40 ms while the fault remained present. Probe windows and timeouts contribute to the delay. The value is not an exact packet-level outage duration.

| Milestone | Traffic time | Meaning |
|---|---|---|
| Fault starts | 10.01 s | Primary link is disconnected. |
| Route changes | 12.11 s | Backup forwarding is installed. |
| Recovery confirmed | 18.03 s | Two healthy application probe windows have been observed. |

# 8 CURRENT METHOD COMPARISON

The same packet-loss configuration ran once with each method. RF + guard and fixed rules restored communication. RF only made no route change and did not verify recovery during the fault.

| Measurement | RF + guard | RF only | Fixed rules |
|---|---|---|---|
| Changed route? | Yes | No | Yes |
| Recovery confirmed | 3.30 s | Not observed | 3.35 s |
| Data received | 3.97 Mbps | 3.63 Mbps | 3.97 Mbps |
| Packets lost | 0.64% | 9.24% | 0.72% |

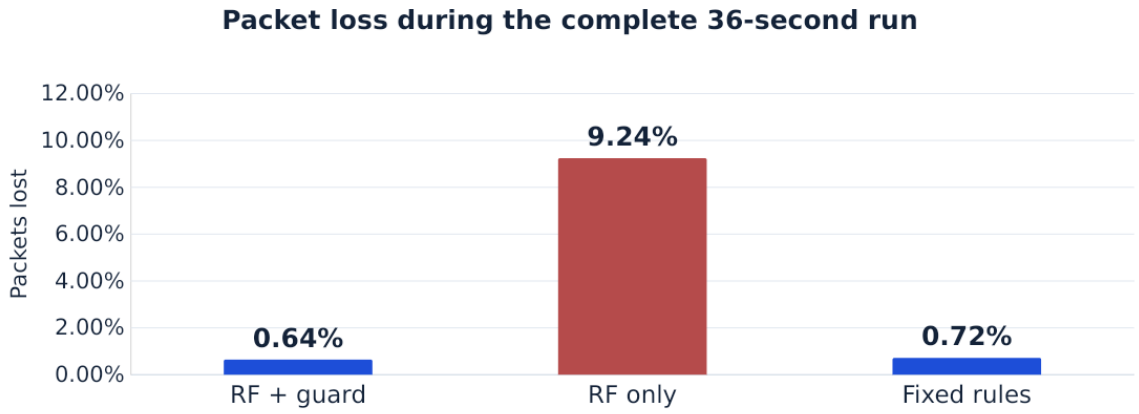



Figure 3 Whole-run application packet loss in the current method comparison

RF alone lost about 9.24% of application packets. Adding the loss guard reduced the observed loss to about 0.64% in the matching guarded run. Fixed rules also recovered, with about 0.72% loss. These results support the safeguard in this tested condition. The small timing difference between guarded RF and fixed rules does not show that one is consistently faster.

# 9 READING THE DASHBOARD GRAPHS

All views use elapsed traffic time. Blue denotes the primary path, grey denotes backup, and pink marks the actual fault window. Path probes continue on their own branches after the application switches.

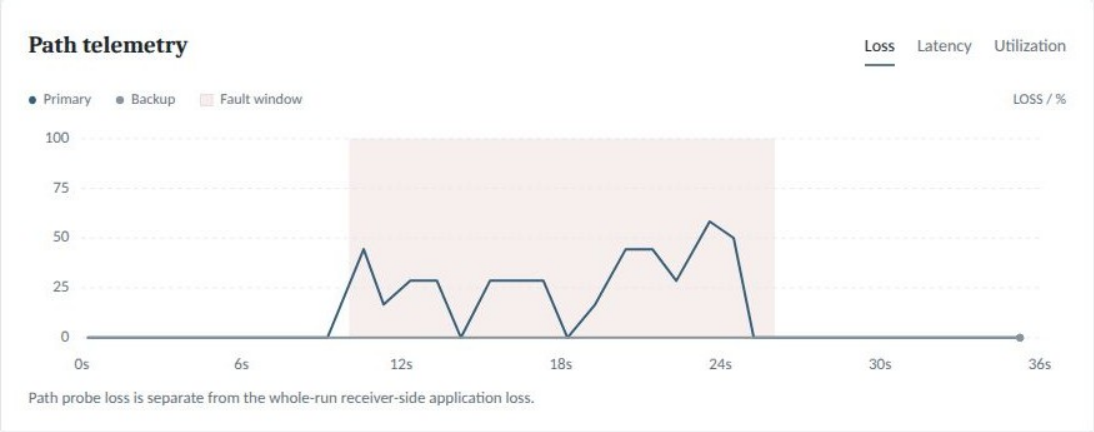



Figure 4 Loss asks how many path probes failed to return

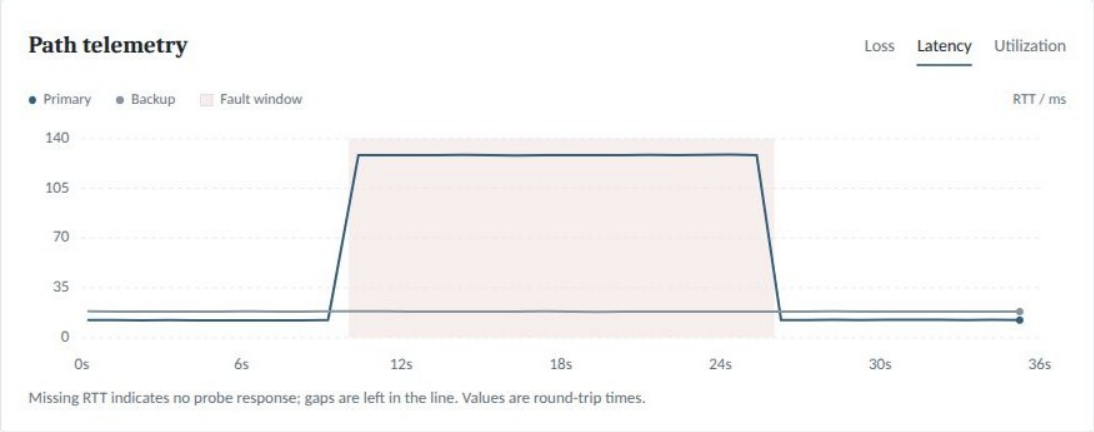



Figure 5 Latency asks how long a complete probe round trip took




Figure 6 Utilization asks how busy the monitored branch link was

# 10 MODEL RELIABILITY

## 10.1 What the forest does

The fitted Random Forest uses loss, RTT, RTT standard deviation, and utilization to classify a path as normal or degraded. It has 200 trees. A degraded score of at least 0.65 marks the path unhealthy. That threshold does not mean 65% accuracy. The model is loaded during a run and does not learn from that demonstration.

## 10.2 What the saved validation score means

The bundled artifact stores a 100% validation result on 544 original lab measurement rows: 481 normal and 63 degraded. Fitting used 1,094 rows. Dedicated packet-loss runs were excluded from fitting. These are stored training-report figures, not new accuracy measurements from the eight current demonstrations.

The original fitting datasets are absent, so the stored score has not been independently reproduced here. Rows from the same run are related, and the tested conditions cover a limited laboratory. A perfect score on those rows does not guarantee correct decisions for unfamiliar faults or networks. Known delay-only inputs can also be missed.

## 10.3 What the safeguards add

| Protection | Purpose |
|---|---|
| Loss of at least 15% | The guard overrides the forest and declares degradation. |
| Separate link-state check | Detects an explicitly disconnected link. |
| Two soft-fault confirmations | Reduces reactions to a single unhealthy reading. |
| Alternative-path check | Avoids deliberately selecting a known unhealthy route. |
| Post-change application probes | Check whether communication actually recovered. |

The packet-loss comparison demonstrates a missed fault by RF alone and a successful guarded response. It does not establish a general superiority of ML. More independent runs and unfamiliar severities, loads, and topologies are needed to estimate reliability.

# 11 CONCLUSION AND NEXT TESTS

## 11.1 Findings from the current runs

The default guarded controller restored application communication in the link-failure, packet-loss, congestion, and delay demonstrations. It retained the primary during healthy operation. When both branches were disconnected, there was no healthy alternative and no credited reroute recovery during the fault.

The packet-loss comparison showed why the loss guard matters in this implementation. RF alone missed the fault. Guarded RF and fixed rules both recovered. Because each condition ran once, the measured timings and loss percentages describe these runs rather than a consistent performance ranking.

## 11.2 What the dashboard makes visible

The network diagram shows the selected route and latest path measurements. Independent probes explain standby RTT and continuing loss on an unused primary. Receiver throughput and application loss describe delivered UDP service across the whole run. Playback and downloadable evidence make these distinctions reviewable.

## 11.3 Traffic above the configured capacity

Offering 11 Mbps to a 10 Mbps route can cause congestion even when no fault is injected. Queues, losses, and limited receiver throughput can result. Moving the same application to another 10 Mbps branch does not combine the two capacities. Keep 4 Mbps for the standard fault demonstrations so load-induced overload does not obscure the injected fault.

## 11.4 Further evaluation

Repeat each condition several times and report variation before claiming one detector is faster. Test more fault severities, traffic loads, capacities, and less similar topologies. Evaluate RF alone separately from its guard, retain a final independent test set, and compare probe-based recovery with more detailed application measurements.

# 12 REFERENCES

[1] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” Proc. HotNets, 2010, doi: 10.1145/1868447.1868466. https://conferences.sigcomm.org/hotnets/2010/papers/a19-lantz.pdf

[2] D. Kreutz et al., “Software-defined networking: A comprehensive survey,” arXiv:1406.0440, 2014. https://arxiv.org/abs/1406.0440

[3] J. Sánchez, I. Grida Ben Yahia, and N. Crespi, “POSTER: Self-healing mechanisms for software-defined networks,” arXiv:1507.02952, 2015. https://arxiv.org/abs/1507.02952

[4] E. H. Makiyah, M. N. Rasool, and F. A. Rawdhan, “A reinforcement learning framework for self-healing fault recovery in intent-based SDNs,” Discover Networks, vol. 2, art. 14, 2026, doi: 10.1007/s44354-026-00028-z.

[5] Mininet Project, “Download/Get started with Mininet,” accessed Oct. 9, 2026. https://mininet.org/download/

[6] Open vSwitch Project, “ovs-actions,” accessed Oct. 9, 2026. https://docs.openvswitch.org/en/latest/ref/ovs-actions.7/

# APPENDIX A DEMONSTRATION GUIDE

### A1 Start the local website

Open a terminal in the project directory and run the following command. Mininet needs administrator access to create the virtual network.

```
sudo python3 dashboard/server.py
```

Open http://127.0.0.1:8765. Select six switches, 4 Mbps traffic, 10 Mbps capacity and Random Forest + loss guard. Disable automatic return to match these recordings. Current defaults enable stable return to primary. Start an experiment and wait for completion; whole-run UDP statistics appear at the end.

For a higher-capacity run: sudo python3 main.py demo --capacity 20 --load 4. This configures both branches at 20 Mbps. Capacity changes do not retrain the RF model or establish its accuracy at new capacities. The saved results in this report remain 10 Mbps observations.

### A2 Explain a successful run

First show the primary branch carrying traffic. Then point out the fault, the controller decision, the route change, and the later recovery confirmation. Explain that the damaged primary can remain unhealthy after the application moves to backup. Use the outcome and application statistics to assess service, rather than the animation alone.

### A3 Replay and compare

Run history opens saved recordings. The replay slider changes the displayed measurement time. Compare methods runs guarded RF, RF only, and fixed rules sequentially under matching settings. It selects the latest matching completed recording for each method, so check the run IDs before comparing figures. Matching includes link capacity; older recordings without a capacity field are treated as 10 Mbps.

### A4 Reset the visible dashboard

After the experiment and any comparison queue finish, run python3 reset_dashboard.py and refresh with Ctrl+Shift+R. This hides older records and preserves their files. Use python3 reset_dashboard.py --restore to show all records again. A server restart and sudo are not required for the reset.

### A5 Presentation sequence

Use normal operation first, then link failure, packet loss, congestion, increased delay, and both paths down. Finish with the packet-loss method comparison. Describe normal recovery as unnecessary and both-down as having no alternative. The current report explains the same eight-run evidence set used in the presentation.

# APPENDIX B CURRENT RUN EVIDENCE

All main results come from the following eight current Mininet runs. The guarded loss run appears in both the scenario table and the method comparison. The table identifies the records; it does not count that shared run twice.

| Scenario | Method | Run ID |
|---|---|---|
| Normal | RF + guard | 20261027T151646Z-776f29cf |
| Link failure | RF + guard | 20261027T151725Z-f6caa845 |
| Packet loss | RF + guard | 20261027T150945Z-5b433df7 |
| Congestion | RF + guard | 20261027T151807Z-9c06ddef |
| Increased delay | RF + guard | 20261027T151849Z-811520ad |
| Both paths down | RF + guard | 20261027T151928Z-180225f4 |
| Packet loss | RF only | 20261027T151025Z-7bbacb9c |
| Packet loss | Fixed rules | 20261027T151105Z-3f4808d8 |

## B1 Files within each run

| File | What it explains |
|---|---|
| metadata.json | Configuration, timestamps, completion status, source and model fingerprints |
| samples.csv | Path measurements, application probes, and controller decisions |
| events.jsonl | Actual fault and route-change timestamps |
| summary.json | The defined final statistics and recovery outcome |
| application-iperf.json | Raw receiver-side UDP measurements |

The files live under .runtime/runs/<run ID>/. Full precision is preserved there. Main tables use two decimals for readability. The graphs and screenshots in this version belong to these runs. No previous campaign averages or earlier-model accuracy figures are presented as current results.

Stored model metadata comes from src/selfheal/assets/detector.joblib. Implementation definitions come from core.py, telemetry.py, learning.py, experiment.py, and dashboard/static/app.js. The earlier report and presentation remain available separately for historical study context.